Day 3 handout

Day 3: Development Tools & Workflows for Scientific Programming

PhD Course in Integrative Neurosciences - Introduction to Scientific Programming

Table of Contents

1. Code Editors and IDEs
2. Notebooks & Interactive Programming
3. Documentation with Markdown and Docstrings
4. Virtual Environments
5. Git Fundamentals and GitHub Workflows
6. Computational Research Workflows

---

1. Code Editors and IDEs

What is the Difference?

Code Editor: A text editor with features specifically designed for writing code:

- Syntax highlighting
- Auto-completion
- Basic file management
- Lightweight and fast

Integrated Development Environment (IDE): A comprehensive software suite that provides:

- All features of a code editor
- Integrated debugger
- Version control integration
- Testing tools
- Project management
- Terminal/console
- Database management
- More resource-intensive

Popular Options for Python Development

Tool	Type	Best For	Pros	Cons
VS Code	Editor	General-purpose, extensible	Free, huge extension ecosystem, Git integration, lightweight	Requires configuration
PyCharm	IDE	Professional Python development	Powerful debugging, testing, refactoring tools	Heavy, paid version for full features
Jupyter Notebook/Lab	Interactive	Data analysis, exploration, teaching	Live code execution, inline visualizations	Not ideal for production code
Spyder	IDE	Scientific computing	Integrated with Anaconda, variable explorer	Less modern interface
Sublime Text	Editor	Fast editing	Very fast, multiple cursors	Limited Python-specific features
Atom	Editor	Customization	Highly customizable, GitHub integration	Slower than alternatives
Vim/Emacs	Editor	Keyboard-centric workflow	Extremely powerful, available everywhere	Steep learning curve

Key Features to Look For

1. **Syntax Highlighting**: Color-codes different parts of your code for readability
2. **Auto-completion**: Suggests code as you type (IntelliSense)
3. **Linting**: Real-time error detection and style checking
4. **Debugging**: Set breakpoints, step through code, inspect variables
5. **Git Integration**: Visual diff, commit, push/pull directly from editor
6. **Extensions/Plugins**: Extend functionality as needed

---

2. Notebooks & Interactive Programming

Introduction to Jupyter Notebooks

Jupyter Notebooks are **web-based interactive computing platforms** that combine:

- **Live code execution**
- **Rich text narratives** (Markdown)
- **Equations** (LaTeX)
- **Rich output** (tables, formatted text, images)
- **Interactive exploration** (cell-by-cell execution)

This makes them ideal for:

- **Exploratory data analysis**

- **Prototyping and testing small pieces of code**
- **Teaching and learning**
- **Sharing research findings**
- **Reproducible research**

Why Jupyter for Scientific Computing?

> "The notebook combines live code, equations, narrative text, visualizations, interactive dashboards and other media." — Project Jupyter

Key Benefits:

1. **Interactive Development**: Execute code cell-by-cell, see results immediately
2. **Rich Output**: Display tables, formatted text, images, videos, LaTeX equations
3. **Documentation**: Mix code with explanations in a single document
4. **Reproducibility**: Share complete analysis workflow
5. **Experimentation**: Quickly test ideas without creating separate files

Installation

In the course environment (recommended): you already have Miniconda and the `scientific-programming` environment from Day 2. Add JupyterLab to it:

```
conda activate scientific-programming
conda install jupyterlab
jupyter lab
```

With pip (venv users):

```
pip install jupyterlab
jupyter lab
```

Classic Notebook interface (optional):

```
pip install notebook
jupyter notebook
```

JupyterLab is the next-generation interface and also includes everything needed to open and run `.ipynb` files.

Jupyter Notebook Interface

Starting Jupyter

When you run `jupyter notebook`, it:

1. Starts a local server (typically at `http://localhost:8888`)
2. Opens your browser to the **Notebook Dashboard**
3. Shows files and directories from the launch location

Creating Your First Notebook

1. Navigate to desired folder in dashboard
2. Click **New** → **Python 3** (or your kernel)
3. A new tab opens with `Untitled.ipynb`
4. Click the title to rename

Understanding Cells

Two Main Cell Types:

1. **Code Cells**: Execute Python code
    - Input label: `In [1]:`
    - Output appears below
    - Number indicates execution order
    - `In [*]:` means currently executing
2. **Markdown Cells**: Formatted text
    - No input label
    - Rendered when executed
    - Can contain headers, lists, links, images, LaTeX

Cell Modes

- **Edit Mode** (green border): Actively typing in a cell
    - Enter edit mode: Press `Enter` or click in cell
    - Can modify cell content
- **Command Mode** (blue border): Navigate and manipulate cells
    - Enter command mode: Press `Esc`
    - Execute keyboard shortcuts

Essential Keyboard Shortcuts

Command Mode (Press `Esc` first)

- **Navigation:**
    - `↑` / `↓`: Move between cells
    - `A`: Insert cell **A**bove
    - `B`: Insert cell **B**elow
- **Cell Type:**
    - `M`: Change to **M**arkdown
    - `Y`: Change to code (p**Y**thon)
- **Cell Operations:**

    - `D`, `D`: **D**elete cell (press twice)
    - `Z`: Undo delete
    - `X`: Cut cell
    - `C`: Copy cell
    - `V`: Paste cell below

- **Running:**

    - `Shift + Enter`: Run cell, select below
    - `Ctrl + Enter`: Run cell, stay in place
    - `Alt + Enter`: Run cell, insert below

Edit Mode (Press `Enter` to activate)

- `Ctrl + Enter`: Run cell
- `Shift + Enter`: Run cell, move to next
- `Alt + Enter`: Run cell, insert new below
- `Ctrl + Shift + -`: Split cell at cursor
- `Tab`: Code completion or indent
- `Shift + Tab`: Show documentation

Working with Code Cells

Basic Execution

```
# Simple output
print("Hello, Jupyter!")

# Last line is automatically displayed
2 + 2

# Import from the standard library
import math

# Code with a simple text "visualization"
for angle in range(0, 361, 45):
    value = math.sin(math.radians(angle))
    bar = '#' * round(abs(value) * 20)
    print(f"{angle:>3} deg  {value:+.2f}  {bar}")
```

The Kernel

What is a Kernel? The kernel is the computational engine that executes your code:

- Each notebook connects to one kernel
- Maintains state across cells (variables, imports persist)
- Execution order matters (use cell numbers to track)

Kernel Operations (Kernel menu):

- **Restart**: Clear all variables, fresh start
- **Restart & Clear Output**: Also removes output

- **Restart & Run All**: Clean slate, execute all cells
- **Interrupt**: Stop a running cell (useful for infinite loops)

Important: Execution Order

```
# Cell 1
x = 10

# Cell 2
print(x)  # Outputs: 10

# Cell 3 (run this, then re-run Cell 2)
x = 100
```

The kernel maintains state! Variables exist until the kernel is restarted, regardless of cell position in the notebook.

Working with Markdown Cells

Markdown cells let you create narrative text with formatting. After typing, press `Shift + Enter` to render.

See the Markdown section below for comprehensive syntax.

Magic Commands

Jupyter includes special commands called "magics":

Line Magics (prefix with `%`)

```
# Time execution of a single line
%timeit sum(range(1000))

# Run external Python file
%run script.py

# List all variables
%whos

# Short list of variable names
%who

# Display system command output
%ls  # or !ls on Unix
```

Cell Magics (prefix with `%%`)

```
%%timeit
# Time execution of entire cell
total = 0
```

```
for i in range(1000):
    total += i
```

```
%%time
# Time a single run of the entire cell
total = sum(i * i for i in range(1_000_000))
print(total)
```

```
%%bash
# Run cell as bash script
echo "Hello from bash"
pwd
```

Rich Display and Visualization

Built-in Display

```
from IPython.display import Image, HTML, Markdown, Math, display

# Display image
display(Image('path/to/image.png'))

# Display HTML
display(HTML('<h2>Formatted HTML</h2>'))

# Display Markdown generated from Python
display(Markdown('**Bold** text and a list:\n\n- first\n- second'))

# Display LaTeX math
display(Math(r'\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i'))
```

Tables from Plain Python Data

A list of dictionaries can be turned into an HTML table and displayed as a formatted table, using only the standard library and `IPython.display`:

```
from statistics import mean
from IPython.display import HTML, display

trials = [
    {"trial": 1, "condition": "A", "reaction_time": 0.52},
    {"trial": 2, "condition": "B", "reaction_time": 0.61},
    {"trial": 3, "condition": "A", "reaction_time": 0.48},
    {"trial": 4, "condition": "B", "reaction_time": 0.66},
]

header = "".join(f"<th>{key}</th>" for key in trials[0])
rows = "".join(
    "<tr>" + "".join(f"<td>{value}</td>" for value in row.values()) +
"</tr>"
    for row in trials
```

```
        )
        display(HTML(f"<table><tr>{header}</tr>{rows}</table>"))

        print("Mean reaction time:", round(mean(t["reaction_time"] for t in
trials), 3))
```

Text Bar Charts

When you only need a quick look at your data, a text bar chart made with # characters works anywhere (including a plain terminal):

```
        import random
        from collections import Counter

        random.seed(42)  # fixed seed: same numbers every run
        rolls = [random.randint(1, 6) for _ in range(60)]
        counts = Counter(rolls)

        for face in range(1, 7):
            print(f"{face} | {'#' * counts[face]} ({counts[face]})")
```

Best Practices for Notebooks

1. Organization

- **One notebook per analysis/topic**
- **Start with imports**: Put all imports in first cell
- **Setup cells**: Configuration in early cells
- **Logical flow**: Top to bottom execution

2. Documentation

- **Title cell**: First Markdown cell with title and overview
- **Section headers**: Use Markdown headers to organize
- **Explain reasoning**: Why you're doing something, not just what
- **Document assumptions**: Make your thinking explicit

3. Code Quality

- **Keep cells focused**: One task per cell when possible
- **Use functions**: Define reusable functions
- **Clear variable names**: Self-documenting code
- **Comments**: Explain complex logic

4. Reproducibility

- **Clear execution order**: Run "Restart & Run All" before sharing
- **Specify dependencies**: Document required packages
- **Include data loading**: Show where data comes from
- **Random seeds**: Set seeds for reproducible randomness

```python
# Good practice: Setup cell at top
import csv
import random
import statistics
from pathlib import Path

# Set random seed for reproducibility
random.seed(42)

# Configuration in one place
DATA_DIR = Path("data")
N_SAMPLES = 100
```

5. Before Sharing

1. **Restart & Run All**: Ensures cells execute in order
2. **Clear unnecessary output**: Remove debugging outputs
3. **Check for errors**: No cells should error
4. **Add explanations**: Would someone else understand?
5. **Save**: Ctrl+S to save

Sharing Notebooks

Export Formats

File → Download as:

- **`.ipynb`**: Native format, can be re-executed
- **`.html`**: Static webpage, preserves formatting
- **`.pdf`**: Via LaTeX, for printing/archiving
- **`.py`**: Extract code only
- **`.md`**: Markdown document

Online Platforms

1. **GitHub**: Automatically renders `.ipynb` files
2. **nbviewer** (https://nbviewer.org/): View GitHub notebooks with better rendering
3. **Google Colab**: Cloud-based, collaborative, free GPUs
4. **Binder**: Launch interactive notebooks from GitHub
5. **JupyterHub**: Shared server for teams

Jupyter Notebook vs JupyterLab

Feature	Jupyter Notebook	JupyterLab
Interface	Single document	Multi-tabbed workspace
File Browser	Separate dashboard	Integrated sidebar
Terminals	Opens in new tab	Integrated
Text Editor	Limited	Full-featured
Extensions	Manual install	Built-in manager

Feature	Jupyter Notebook	JupyterLab
Layout	Fixed	Flexible (split views)
Best For	Beginners, simple projects	Power users, complex projects

When to use JupyterLab:

- Working with multiple notebooks simultaneously
- Need integrated terminal access
- Editing code files alongside notebooks
- Complex projects requiring multiple tools

When to use Jupyter Notebook:

- Learning Python/data science
- Single-focus analysis
- Teaching (simpler interface)
- Lightweight, quick tasks

Common Pitfalls and Solutions

Problem: Cells Execute Out of Order

Solution: Use "Restart & Run All" frequently to verify linear execution

Problem: Kernel Dies or Hangs

Solutions:

- Save work frequently
- Interrupt kernel (Kernel → Interrupt)
- Restart kernel if unresponsive
- Check for infinite loops or memory issues

Problem: Large Outputs Slow Down Notebook

Solutions:

- Clear output: Cell → All Output → Clear
- Suppress output: End cell with semicolon `;`
- Write large outputs to file instead

Problem: Variables from Deleted Cells Still Exist

Solution: Restart kernel to clear all variables

---

3. Documentation with Markdown and Docstrings

Why Documentation Matters

> "Code is more often read than written." — Guido van Rossum (Creator of Python)

Good documentation:

- **Helps future you** understand your code
- **Enables collaboration** with others
- **Facilitates reuse** of your work
- **Increases impact** of your research

Markdown: Comprehensive Syntax Guide

Markdown is a lightweight markup language for creating formatted text using plain text.

Headers

```
# Heading 1 (Largest)
## Heading 2
### Heading 3
#### Heading 4
##### Heading 5
###### Heading 6 (Smallest)
```

Emphasis

```
*Italic text* or _italic text_

**Bold text** or __bold text__

***Bold and italic*** or ___bold and italic___

~~Strikethrough text~~
```

Renders as:

- *Italic text*
- **Bold text**
- ***Bold and italic***
- ~~Strikethrough~~

Lists

Unordered Lists:

```
* Item 1
* Item 2
  * Subitem 2a
```

```
  * Subitem 2b
* Item 3
```

Or use + or - instead of *.

Ordered Lists:

```
1. First item
2. Second item
   1. Subitem 2.1
   2. Subitem 2.2
3. Third item
```

Nested Lists:

```
1. First level
   - Second level
     - Third level
       - Fourth level
```

Links

```
[Link text](https://example.com)

[Link with title](https://example.com "This is a tooltip")

# Reference-style links
[Link text][reference]

[reference]: https://example.com "Optional title"
```

Images

```
![Alt text](path/to/image.png)

![Image with title](path/to/image.png "Image title")

# Reference-style
![Alt text][image-ref]

[image-ref]: path/to/image.png "Optional title"
```

Code

Inline Code:

```
Use `code` for inline code snippets.
```

Code Blocks:

```
```python
def hello():
```
```

````
    print("Hello, World!")
```
````

Specify language for syntax highlighting: `python`, `bash`, `r`, `javascript`, etc.

Blockquotes

```
> This is a blockquote.
> It can span multiple lines.
>
> > Nested blockquote
```

Renders as:

> This is a blockquote. It can span multiple lines.

Horizontal Rules

```
---
or
***
or
___
```

Tables

```
| Header 1 | Header 2 | Header 3 |
|----------|----------|----------|
| Row 1    | Data     | Data     |
| Row 2    | Data     | Data     |

# Alignment
| Left | Center | Right |
|:-----|:------:|------:|
| L    |   C    |     R |
```

Renders as:

Header 1	Header 2	Header 3
Row 1	Data	Data

LaTeX Math

In Jupyter notebooks and many Markdown processors:

Inline Math:

```
Einstein's equation is $E = mc^2$.
```

Display Math:
```

```
$$
\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi}
$$
```

Common Math Symbols:

```
$\alpha, \beta, \gamma, \Delta, \Sigma$

$x^2, x_i, x_{ij}$

$\frac{a}{b}, \sqrt{x}, \sqrt[n]{x}$

$\sum_{i=1}^{n} x_i, \int_0^1 f(x)dx$

$\mathbf{v}, \hat{x}, \bar{x}, \tilde{x}$
```

Task Lists (GitHub Flavored Markdown)

```
- [x] Completed task
- [ ] Incomplete task
- [ ] Another incomplete task
```

Escaping Characters

Use backslash \ to escape special characters:

```
\* Not a bullet point
\# Not a header
\[Not a link\](url)
```

HTML in Markdown

You can include HTML for advanced formatting:

```
<span style="color:red">Red text</span>

<details>
<summary>Click to expand</summary>

Hidden content here

</details>
```

Footnotes

```
Here's a sentence with a footnote.[^1]

[^1]: This is the footnote content.
```

Python Docstrings

Docstrings are string literals that appear as the first statement in a module or a function (and, later in the course, in classes and methods, which have not been introduced yet). They document what your code does.

Why Use Docstrings?

1. **Accessible via `help()`**: Users can get help interactively
2. **Auto-documentation**: Tools like Sphinx can generate docs
3. **Better IDEs**: Provides hover information and auto-completion
4. **Maintainability**: Future developers (including you!) understand intent

Basic Docstring Format

```
        def function_name(parameter):
            """
            One-line summary of function.

            More detailed description if needed. Explain what the function
does,
            not how it does it (that's what the code shows).

            Parameters
            ----------
            parameter : type
                Description of parameter.

            Returns
            -------
            return_type
                Description of return value.

            Examples
            --------
            >>> function_name(10)
            20
            """
            return parameter * 2
```

Docstring Conventions (PEP 257)

1. **Triple double quotes**: Always use `"""`
2. **One-line summary**: Brief, imperative mood ("Calculate..." not "Calculates...")
3. **Blank line**: After summary if multi-line
4. **Detailed description**: What it does, not how
5. **Ends with blank line**: Before closing quotes (multi-line only)

Function/Method Docstrings

Simple Function:

```python
def add(a, b):
    """Return the sum of a and b."""
    return a + b
```

Complex Function (Sectioned Style):

```python
import statistics


def calculate_statistics(data, method='mean'):
    """
    Calculate a statistical measure for the given data.

    This function computes the mean, median, or standard deviation
    of a list of numbers.

    Parameters
    ----------
    data : list of float
        Input values. Must contain at least two values for 'std'.
    method : {'mean', 'median', 'std'}, optional
        Statistical method to compute. Default is 'mean'.

    Returns
    -------
    result : float
        The computed statistic.

    Raises
    ------
    ValueError
        If method is not one of the allowed values.
    TypeError
        If data contains non-numeric values.

    See Also
    --------
    statistics.mean : Arithmetic mean
    statistics.median : Median value
    statistics.stdev : Sample standard deviation

    Notes
    -----
    The standard deviation is the sample standard deviation
    (n - 1 in the denominator).

    Examples
    --------
    >>> calculate_statistics([1, 2, 3, 4, 5], method='mean')
    3
    >>> calculate_statistics([1, 2, 3, 4], method='median')
    2.5
```

```
    """
    if method not in ('mean', 'median', 'std'):
        raise ValueError(f"Unknown method: {method!r}")
    if not all(isinstance(x, (int, float)) for x in data):
        raise TypeError("data must contain only numbers")

    if method == 'mean':
        return statistics.mean(data)
    if method == 'median':
        return statistics.median(data)
    return statistics.stdev(data)
```

The `>>>` lines in the `Examples` section are not just decoration: the standard library module `doctest` can run them and check that the documented output is still correct (`python -m doctest my_module.py`).

> **Note on classes:** docstrings are also written for classes and their methods. Classes are introduced later in the course, so here we only use module-level and function-level docstrings.

Module Docstrings

Place at the very beginning of the file:

```
"""
Data Analysis Utilities
=======================

This module provides utility functions for summarizing experimental
measurements, including basic descriptive statistics.

Available Functions
-------------------
summarize : Compute mean, median and standard deviation

Dependencies
------------
Python standard library only (statistics, collections).

Examples
--------
>>> import data_utils
>>> data_utils.summarize([1, 2, 3, 4, 5])
{'mean': 3, 'median': 3, 'std': 1.5811388300841898}

Notes
-----
All functions expect plain Python lists of numbers.

Author: Your Name
License: MIT
"""
```

```
        import statistics
        from collections import Counter


        def summarize(values):
            """Return the mean, median and standard deviation of a list of
numbers."""
            return {
                'mean': statistics.mean(values),
                'median': statistics.median(values),
                'std': statistics.stdev(values),
            }
```

Docstring Formats Comparison

Google Style:

```
        def function(arg1, arg2):
            """
            Summary line.

            Extended description.

            Args:
                arg1 (int): Description of arg1
                arg2 (str): Description of arg2

            Returns:
                bool: Description of return value

            Raises:
                ValueError: If arg1 is negative
            """
```

Sectioned Style (underlined section headings):

```
        def function(arg1, arg2):
            """
            Summary line.

            Extended description.

            Parameters
            ----------
            arg1 : int
                Description of arg1
            arg2 : str
                Description of arg2

            Returns
            -------
```

```
        bool
            Description of return value

        Raises
        ------
        ValueError
            If arg1 is negative
        """
```

reStructuredText Style:

```
    def function(arg1, arg2):
        """
        Summary line.

        Extended description.

        :param arg1: Description of arg1
        :type arg1: int
        :param arg2: Description of arg2
        :type arg2: str
        :return: Description of return value
        :rtype: bool
        :raises ValueError: If arg1 is negative
        """
```

Recommendation for Scientific Python: Use the **sectioned style** (the underlined `Parameters` / `Returns` headings), as it is widely used in scientific Python packages. Whichever style you choose, use it consistently within a project.

Accessing Docstrings

```
    # View in interactive session
    help(function_name)

    # Access docstring programmatically
    print(function_name.__doc__)

    # In Jupyter/IPython
    function_name?     # Quick info
    function_name??    # Source code + docstring
```

Documentation Best Practices

1. **Be concise but complete**: Include necessary information without verbosity
2. **Use examples**: Show how to use the function
3. **Document parameters**: Type and description for each
4. **Explain return values**: What is returned and when
5. **List exceptions**: What can go wrong
6. **Update documentation**: When code changes, update docs
7. **Use consistent style**: Pick one docstring format and stick to it

---

4. Virtual Environments

Why Virtual Environments?

The Problem:

- Different projects need different package versions
- System Python is used by your OS (don't mess with it!)
- Need to share exact dependencies with collaborators
- Want isolated, reproducible environments

The Solution: Virtual Environments

A virtual environment is an isolated Python installation with its own:

- Python interpreter
- Package library (site-packages)
- Scripts and executables

> **You already use one.** The `scientific-programming` environment from Day 2 is a conda environment created from a file, `environment-base.yml` (its pip twin is `requirements-base.txt`). Open both files: they are small real examples of what this section teaches. Later days extend the same environment with `conda env update`.

Two Main Tools: venv and conda

Feature	venv	conda
Built-in	Yes (Python 3.3+)	No
Installation	None needed	Install Miniconda/Anaconda
Python versions	Uses system Python	Manages Python versions
Non-Python packages	No	Yes (R, C libraries)
Speed	Fast	Slower
Ecosystem	PyPI only	conda-forge, defaults
Scientific packages	Limited	Excellent
Complexity	Simple	Moderate
Best for	Pure Python projects	Data science, research

Other tools exist (see Optional: other environment tools below), but venv and conda are the two you need to know.

Python venv (Built-in)

Creating Virtual Environments

```
# Create virtual environment in current directory
python -m venv myenv

# Create with specific name
```

```
python -m venv project_env

# Create with specific Python version (if multiple installed)
python3.11 -m venv myenv
```

This creates a directory (`myenv/`) containing:

- Python interpreter copy
- pip
- Empty `site-packages/` directory
- Activation scripts

Activating Virtual Environments

Linux/Mac:

```
source myenv/bin/activate
```

Windows (Command Prompt):

```
myenv\Scripts\activate.bat
```

Windows (PowerShell):

```
myenv\Scripts\Activate.ps1
```

When activated, you'll see:

```
(myenv) user@computer:~$
```

Using Virtual Environments

```
# Activate environment
source myenv/bin/activate

# Install packages
pip install requests

# List installed packages
pip list

# Create requirements file
pip freeze > requirements.txt

# Deactivate environment
deactivate
```

Reproducing Environments

```
# Create new environment
python -m venv newenv
source newenv/bin/activate
```

```
# Install from requirements
pip install -r requirements.txt
```

Deleting Virtual Environments

```
# Deactivate first
deactivate

# Remove directory
rm -rf myenv
```

Conda

Conda is a package manager and environment manager combined, particularly strong for scientific computing.

Installing Conda

Miniconda (Minimal, Recommended; already installed if you followed Day 2):

```
# Download from https://docs.conda.io/en/latest/miniconda.html
# Install and follow prompts

# Verify installation
conda --version
```

Anaconda (Full distribution with 250+ packages):

- Download from https://www.anaconda.com/download/
- Larger install (~5GB) but includes most scientific packages

Creating Conda Environments

```
# Create environment with specific Python version
conda create -n myenv python=3.11

# Create with packages
conda create -n webenv python=3.11 requests

# Create from environment file
conda env create -f environment.yml
```

Activating/Deactivating

```
# Activate
conda activate myenv

# Deactivate
conda deactivate
```

```
# List environments
conda env list
```

Managing Packages

```
# Install package
conda install requests

# Install specific version
conda install requests=2.31.0

# Install from specific channel
conda install -c conda-forge requests

# Install multiple packages
conda install requests jupyterlab

# Update package
conda update requests

# Remove package
conda remove requests

# List installed packages
conda list
```

Exporting/Importing Environments

Method 1: environment.yml (Recommended)

```
# Export to environment.yml
conda env export > environment.yml

# Create environment from file
conda env create -f environment.yml
```

Example `environment.yml`:

```
name: myproject
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.11
  - requests=2.31
  - pip
```

Method 2: Explicit spec file (Exact reproduction)

```
# Export explicit spec
conda list --explicit > spec-file.txt
```

```
# Create from spec file
conda create --name myenv --file spec-file.txt
```

Method 3: Cross-platform requirements

```
# Export without builds (more portable)
conda env export --from-history > environment.yml
```

Removing Environments

```
# Remove environment
conda env remove -n myenv

# Or
conda remove -n myenv --all
```

Conda Configuration

```
# Set default channel
conda config --add channels conda-forge

# Set channel priority
conda config --set channel_priority strict

# Show configuration
conda config --show
```

Optional: other environment tools

This section is optional. venv and conda are all you need for this course; the tools below are mentioned so that you recognize them when you meet them.

- **uv**: a very fast, pip-compatible installer and environment manager written in Rust. Install it with `pip install uv`, then use it as a drop-in replacement for pip and venv:

```
uv venv myenv                       # create an environment
source myenv/bin/activate           # activate it (Linux/Mac)
uv pip install requests             # install a package
uv pip install requests==2.31.0     # install a specific version
uv pip install -r requirements.txt  # install from a requirements file
```

- **Poetry**: manages dependencies and packaging through a `pyproject.toml` file and creates a lock file with exact versions (`poetry add requests`, `poetry install`). Mostly used for developing packages that you want to publish.
- **Pipenv**: combines pip and venv, tracking dependencies in a `Pipfile` and `Pipfile.lock` (`pipenv install requests`, `pipenv shell`).
- **virtualenvwrapper**: a set of shell commands that keeps all environments in one place and makes them quick to create and switch (`mkvirtualenv myenv`, `workon myenv`, `deactivate`).

Choosing the Right Tool

Use venv when:

- Working on pure Python projects
- Want simplicity and minimal setup
- Don't need non-Python dependencies
- Deploying to production servers

Use conda when:

- Doing data science or scientific computing
- Need non-Python dependencies (GDAL, HDF5, etc.)
- Need different Python versions easily
- Working with bioinformatics packages
- Teaching (Anaconda provides consistent environment)

Best Practices

1. **One environment per project**: Don't mix projects
2. **Document dependencies**: Always maintain requirements file
3. **Never commit environments**: Add to `.gitignore`
4. **Activate before work**: Always activate environment first
5. **Pin versions**: Specify versions for reproducibility
6. **Update regularly**: Keep packages updated (test first!)
7. **Use environment files**: `requirements.txt` or `environment.yml`

Common Workflows

Starting a New Project

With venv:

```
mkdir myproject
cd myproject
python -m venv .venv
source .venv/bin/activate
pip install requests
pip freeze > requirements.txt
git init
echo ".venv/" >> .gitignore
```

With conda:

```
mkdir myproject
cd myproject
conda create -n myproject python=3.11
conda activate myproject
conda install requests
conda env export > environment.yml
```

```
git init
echo "environment file ready for version control"
```

Joining Existing Project

With venv:

```
git clone https://github.com/user/project.git
cd project
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

With conda:

```
git clone https://github.com/user/project.git
cd project
conda env create -f environment.yml
conda activate project
```

Sharing Your Project

Create requirements.txt:

```
# venv
pip freeze > requirements.txt

# Or manually curate (better)
echo "requests>=2.31.0" > requirements.txt
```

Create environment.yml:

```
# conda
conda env export --from-history > environment.yml
```

---

5. Git Fundamentals and GitHub Workflows

Introduction to Version Control

Version Control System (VCS): Software that tracks changes to files over time, allowing you to:

- Recall specific versions later
- Compare changes over time
- See who modified what and when
- Recover from mistakes
- Collaborate without conflicts

Git is the most popular distributed version control system. **GitHub** is a web-based platform for hosting Git repositories.

Why Use Git for Research?

1. **Track changes**: Complete history of your project
2. **Experimentation**: Try ideas without fear (can always revert)
3. **Collaboration**: Multiple people can work simultaneously
4. **Backup**: Distributed nature provides redundancy
5. **Reproducibility**: Tag specific versions used in publications
6. **Transparency**: Clear record of what changed and why

Git Concepts

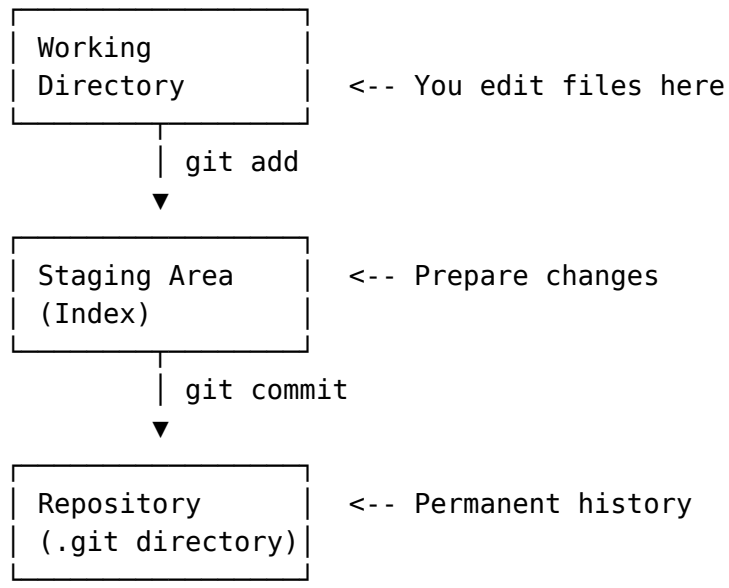
The Three States

Files in Git can be in three states:

1. **Modified**: Changed but not committed
2. **Staged**: Marked for inclusion in next commit
3. **Committed**: Safely stored in repository

```
Working Directory  →  Staging Area  →  Repository
    (modified)         (staged)        (committed)
```

The Git Workflow



Installing Git

Linux (Debian/Ubuntu):

```
sudo apt-get install git
```

Mac:

```
# Via Homebrew
brew install git
```

```
# Or via Xcode Command Line Tools
xcode-select --install
```

Windows:

- Download from https://git-scm.com/download/win
- Or use Git for Windows

Verify installation:

```
git --version
```

Git Configuration

First time setup:

```
# Set your identity
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# Set default editor
git config --global core.editor "code --wait"  # VS Code
# or
git config --global core.editor "nano"  # Nano

# Set default branch name
git config --global init.defaultBranch main

# View configuration
git config --list

# View specific setting
git config user.name
```

Configuration levels:

- `--system`: All users on system
- `--global`: All repositories for your user
- `--local`: Specific repository (default)

Authenticating with GitHub

GitHub no longer accepts your account password for `git push` or `git pull` over HTTPS. Use one of:

1. **GitHub CLI (simplest, all systems):** install it from https://cli.github.com/, then run `gh auth login`, choose **GitHub.com**, **HTTPS**, and **Login with a web browser**. Git then uses this login automatically.
2. **Git Credential Manager (Windows):** installed with Git for Windows. The first push opens a browser window to sign in.

3. **Personal access token:** on GitHub go to Settings, Developer settings, Personal access tokens, create a token with the `repo` scope, and paste it when Git asks for a password. Treat it like a password and never commit it.
4. **SSH keys:** more setup, but no repeated prompts (see GitHub's documentation, "Connecting to GitHub with SSH").

Check that it works with `git push` on your first repository (Exercise 4, Part A). A `Permission denied` or `Authentication failed` message means the login is missing.

Creating a Repository

Option 1: Start from Scratch

```
# Create project directory
mkdir my_project
cd my_project

# Initialize Git repository
git init

# Check status
git status
```

Option 2: Clone Existing Repository

```
# Clone from GitHub
git clone https://github.com/username/repository.git

# Clone into specific directory
git clone https://github.com/username/repository.git my_folder

# Clone specific branch
git clone -b branch_name https://github.com/username/repository.git
```

Basic Git Commands

Checking Status

```
# See current status
git status

# Brief status
git status -s
```

Output example:

```
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
```

```
  modified:   analysis.py

Untracked files:
  results.csv
```

Adding Files (Staging)

```
# Add specific file
git add filename.py

# Add multiple files
git add file1.py file2.py

# Add all files in directory
git add directory/

# Add all changes
git add .

# Add all Python files
git add *.py

# Interactive staging
git add -i
```

Committing Changes

```
# Commit staged changes
git commit -m "Descriptive commit message"

# Commit with multi-line message
git commit -m "Short summary
>
> Detailed explanation of the changes
> and why they were necessary."

# Add and commit in one step (tracked files only)
git commit -am "Update analysis script"

# Open editor for commit message
git commit
```

Good Commit Messages:

```
# Good ✓
git commit -m "Add function to calculate sample statistics"
git commit -m "Fix off-by-one error in loop iteration"
git commit -m "Update README with installation instructions"

# Bad ✗
git commit -m "Update"
```

```
git commit -m "Fix bug"
git commit -m "Changes"
```

Viewing History

```
# View commit history
git log

# Compact view
git log --oneline

# With graph
git log --graph --oneline --all

# Last N commits
git log -n 5

# With file changes
git log --stat

# Search commits
git log --grep="fix"

# By author
git log --author="Your Name"

# By date
git log --since="2 weeks ago"
git log --until="YYYY-MM-DD"
```

Viewing Changes

```
# See unstaged changes
git diff

# See staged changes
git diff --staged

# Compare with specific commit
git diff commit_hash

# Compare two commits
git diff commit1 commit2

# Changes in specific file
git diff filename.py

# Word-level diff (useful for prose)
git diff --word-diff
```

Undoing Changes

```
# Discard changes in working directory
git restore filename.py

# Unstage file (keep changes)
git restore --staged filename.py

# Discard all local changes (dangerous!)
git restore .

# Amend last commit (before pushing)
git commit --amend -m "New message"

# Revert commit (creates new commit)
git revert commit_hash
```

Branches

Branches allow you to work on different features or experiments independently.

```
A---B---C  (main)
     \
      D---E  (feature)
```

Branch Commands

```
# List branches
git branch

# List all branches (including remote)
git branch -a

# Create new branch
git branch feature-name

# Switch to branch
git switch feature-name
# or older syntax:
git checkout feature-name

# Create and switch in one step
git switch -c feature-name
# or:
git checkout -b feature-name

# Rename current branch
git branch -m new-name

# Delete branch (if merged)
git branch -d feature-name
```

```
# Force delete (even if not merged)
git branch -D feature-name

# See which branches are merged
git branch --merged
```

Merging Branches

```
# Switch to branch you want to merge INTO
git switch main

# Merge feature branch into main
git merge feature-name

# Merge with message
git merge feature-name -m "Merge feature: add analysis module"
```

Merge scenarios:

1. **Fast-forward merge** (no conflicts):

```
Before:
    A---B---C (main)
         \
          D---E (feature)

After git merge feature:
    A---B---C---D---E (main)
```

2. **Three-way merge** (diverged branches):

```
Before:
    A---B---C---F (main)
         \
          D---E (feature)

After git merge feature:
    A---B---C---F---G (main, merge commit)
         \         /
          D---E---- (feature)
```

Handling Merge Conflicts

Conflicts occur when same lines are changed in both branches.

```
# Attempt merge
git merge feature-name

# Git indicates conflict:
# Auto-merging analysis.py
# CONFLICT (content): Merge conflict in analysis.py
```

```
# Automatic merge failed; fix conflicts and then commit the result.

# Check status
git status

# Open conflicted file - you'll see:

<<<<<<< HEAD
# Current branch (main) version
result = calculate_mean(data)
=======
# Incoming branch (feature) version
result = calculate_median(data)
>>>>>>> feature-name
```

Resolve conflict:

1. Edit file to keep desired version
2. Remove conflict markers (<<<<<<<, \=\=\=\=\=\=\=, >>>>>>>)
3. Stage resolved file: `git add analysis.py`
4. Complete merge: `git commit`

```
# Or abort merge
git merge --abort
```

Remote Repositories

A remote repository is hosted on GitHub, GitLab, Bitbucket, etc.

Working with Remotes

```
# List remotes
git remote

# Verbose list (shows URLs)
git remote -v

# Add remote
git remote add origin https://github.com/username/repo.git

# Remove remote
git remote remove origin

# Rename remote
git remote rename origin upstream

# Change remote URL
git remote set-url origin https://new-url.git
```

Pushing Changes

```
# Push to remote
git push origin main

# Push all branches
git push origin --all

# Push and set upstream (first time)
git push -u origin main

# After -u, can just use:
git push

# Force push (dangerous! overwrites remote)
git push --force
# Safer alternative:
git push --force-with-lease
```

Fetching and Pulling

```
# Fetch changes (doesn't merge)
git fetch origin

# Pull changes (fetch + merge)
git pull origin main

# Pull with rebase instead of merge
git pull --rebase origin main

# After setting upstream:
git pull
```

Difference:

- `git fetch`: Downloads changes but doesn't modify your working directory
- `git pull`: Downloads and merges changes (`fetch` + `merge`)

GitHub Workflows

Creating a GitHub Repository

On GitHub:

1. Click "New repository"
2. Enter repository name
3. Add description (optional)
4. Choose Public or Private
5. Initialize with README (optional)
6. Add .gitignore (select Python)
7. Choose license (optional)

8. Click "Create repository"

Connect local repository:

New repository:

```
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/username/repo.git
git push -u origin main
```

Existing repository:

```
git remote add origin https://github.com/username/repo.git
git push -u origin main
```

Typical Workflow

```
# 1. Clone repository (once)
git clone https://github.com/username/repo.git
cd repo

# 2. Create feature branch
git switch -c add-analysis

# 3. Make changes
# ... edit files ...

# 4. Check what changed
git status
git diff

# 5. Stage changes
git add analysis.py
git add README.md

# 6. Commit
git commit -m "Add statistical analysis module"

# 7. Push to GitHub
git push -u origin add-analysis

# 8. Create Pull Request on GitHub

# 9. After PR is merged, update local main
git switch main
git pull origin main

# 10. Delete feature branch
git branch -d add-analysis
```

Pull Requests (PRs)

Pull Requests are a GitHub feature (not Git) for:

- Proposing changes
- Code review
- Discussion
- Checking before merge

Creating a PR:

1. Push branch to GitHub
2. Go to repository on GitHub
3. Click "Pull requests" → "New pull request"
4. Select base branch (usually `main`) and compare branch (your feature)
5. Review changes
6. Add title and description
7. Click "Create pull request"

PR Description Template:

```
## Description
Brief description of changes

## Motivation
Why is this change needed?

## Changes Made
- Added function X
- Fixed bug in Y
- Updated documentation

## Checking
How did you check that the change works?

## Screenshots (if applicable)

## Checklist
- [ ] Code follows style guidelines
- [ ] Documentation updated
- [ ] Code runs without errors
```

Forking Workflow

Used when you don't have write access to a repository.

```
# 1. Fork repository on GitHub (click "Fork" button)

# 2. Clone YOUR fork
git clone https://github.com/YOUR_USERNAME/repo.git
cd repo

# 3. Add original as "upstream"
```

```
git remote add upstream https://github.com/ORIGINAL_OWNER/repo.git

# 4. Create feature branch
git switch -c my-feature

# 5. Make changes and commit
git add .
git commit -m "Add feature"

# 6. Push to YOUR fork
git push origin my-feature

# 7. Create Pull Request from your fork to original

# 8. Keep your fork updated
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

.gitignore

`.gitignore` file specifies files Git should ignore.

Python `.gitignore` template:

```
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
env/
venv/
ENV/
.venv
*.egg-info/
dist/
build/

# Jupyter
.ipynb_checkpoints
*.ipynb_checkpoints/

# Data files (if large)
*.csv
*.xlsx
*.h5
*.hdf5
data/
```

```
# Results
results/
output/
figures/

# IDEs
.vscode/
.idea/
*.swp
*.swo
*~

# OS
.DS_Store
Thumbs.db

# Environment
.env
.env.local

# Logs
*.log
```

GitHub Features

Releases and Tags

Tags mark specific points in history (like versions).

```
# Create tag
git tag v1.0.0

# Tag with message
git tag -a v1.0.0 -m "Release version 1.0.0"

# List tags
git tag

# Push tags to GitHub
git push origin v1.0.0

# Push all tags
git push origin --tags

# Create release on GitHub from tag
```

GitHub Issues

Use for:

- Bug reports

- Feature requests
- Task tracking
- Discussion

Issue Template:

```
## Bug Report

**Describe the bug**
Clear description

**To Reproduce**
Steps to reproduce:
1. Run script `analysis.py`
2. With input file `data.csv`
3. See error

**Expected behavior**
What you expected to happen

**Actual behavior**
What actually happened

**Environment**
- OS: Ubuntu 22.04
- Python: 3.10.5
- Package versions: (pip list output)

**Additional context**
```

GitHub Actions (CI/CD)

GitHub Actions can automatically run checks every time you push or open a pull request. Automated testing is covered later in the course.

Best Practices

1. **Commit often**: Small, logical commits
2. **Write clear messages**: Explain why, not just what
3. **Use branches**: Keep main stable, experiment in branches
4. **Pull before push**: Avoid conflicts
5. **Review before committing**: Use `git diff`
6. **Don't commit secrets**: API keys, passwords
7. **Use .gitignore**: Don't track generated files
8. **Run your code before pushing**: Ensure it works
9. **Meaningful branch names**: `fix-bug-123`, `add-feature-x`
10. **Keep commits atomic**: One logical change per commit

Common Git Scenarios

Scenario: Made changes but need to switch branches

```
# Save current work without committing
git stash

# Switch branches
git switch other-branch

# Later, return and restore work
git switch original-branch
git stash pop
```

Scenario: Accidentally committed to wrong branch

```
# On wrong-branch
git switch correct-branch
git cherry-pick <commit-hash>

# Switch back and remove from wrong branch
git switch wrong-branch
git reset --hard HEAD~1
```

Scenario: Need to undo last commit but keep changes

```
# Undo commit, keep changes staged
git reset --soft HEAD~1

# Undo commit, keep changes unstaged
git reset HEAD~1

# Undo commit, discard changes (dangerous!)
git reset --hard HEAD~1
```

Scenario: Merge conflicts during pull

```
git pull origin main

# CONFLICT in file.py

# Option 1: Resolve manually
nano file.py  # Edit file
git add file.py
git commit

# Option 2: Use theirs or ours
git checkout --theirs file.py  # Take remote version
git checkout --ours file.py    # Keep local version
```

```
git add file.py
git commit
```

Scenario: Want to see what changed in last commit

```
git show

# Or specific commit
git show <commit-hash>
```

Visual Guide to Git Workflow

```
┌─────────────────────────────────────────────────────────────┐
│                      DAILY WORKFLOW                         │
└─────────────────────────────────────────────────────────────┘

1. Start day
   ↓
   git pull origin main

2. Create feature branch
   ↓
   git switch -c feature-name

3. Work and commit frequently
   ↓
   [Edit files]
   git add file.py
   git commit -m "Message"
   [Repeat]

4. Push to GitHub
   ↓
   git push -u origin feature-name

5. Create Pull Request
   ↓
   [On GitHub web interface]

6. Code review and merge
   ↓
   [Team reviews, approves, merges]

7. Update local main
   ↓
   git switch main
   git pull origin main

8. Delete feature branch
   ↓
```

```
   git branch -d feature-name

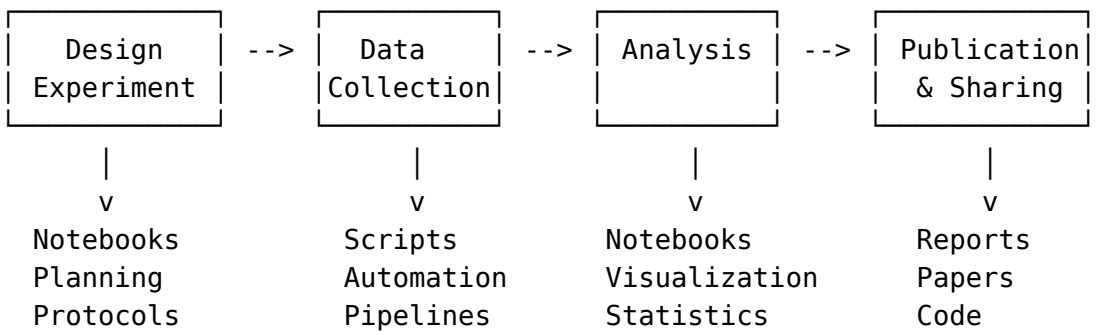
9. Repeat!
```

---

6. Computational Research Workflows

The Research Pipeline

A typical computational research project involves:



Project Organization

Recommended Structure

```
project_name/
│
├── README.md              # Project overview
├── LICENSE                # Software license
├── requirements.txt       # Python dependencies (or environment.yml)
├── .gitignore            # Git ignore rules
│
├── data/                  # Data directory
│   ├── raw/              # Original, immutable data
│   ├── processed/        # Cleaned, transformed data
│   └── README.md         # Data documentation
│
├── notebooks/            # Jupyter notebooks
│   ├── 01_exploration.ipynb
│   ├── 02_analysis.ipynb
│   └── 03_reports.ipynb
│
├── src/                  # Source code
│   ├── __init__.py
│   ├── data_processing.py
│   ├── analysis.py
│   └── reporting.py
│
├── tests/                # Automated tests (covered later in the course)
```

```
│   ├── test_data_processing.py
│   └── test_analysis.py
│
├── scripts/              # Standalone scripts
│   ├── download_data.sh
│   └── run_pipeline.py
│
├── docs/                 # Documentation
│   ├── methods.md
│   └── api.md
│
├── figures/              # Generated figures
│   └── .gitkeep
│
└── results/              # Analysis results
    └── .gitkeep
```

Workflow Integration

1. Project Initialization

```bash
# Create project structure
mkdir myproject
cd myproject

# Initialize Git
git init
git add .gitignore README.md
git commit -m "Initial commit"

# Create virtual environment
python -m venv .venv
source .venv/bin/activate

# Install dependencies
pip install requests jupyterlab

# Save dependencies
pip freeze > requirements.txt

# Create GitHub repository and push
git remote add origin https://github.com/username/myproject.git
git push -u origin main
```

2. Development Workflow

```bash
# Start working session
cd myproject
source .venv/bin/activate
git pull origin main
```

```
# Create feature branch
git switch -c data-processing

# Start Jupyter
jupyter lab

# Work in notebook, develop code

# Extract functions to scripts
# Move stable code from notebook to src/

# Commit regularly
git add notebooks/01_exploration.ipynb
git add src/data_processing.py
git commit -m "Add data loading and cleaning functions"

# Push to GitHub
git push -u origin data-processing

# Create Pull Request when ready
```

3. From Notebook to Script

Start in Notebook (exploratory):

```
# In notebook cell
import csv
import math

# Load data
with open('../data/raw/experiment.csv') as f:
    rows = list(csv.DictReader(f))

# Clean data: keep rows with a valid, positive value
clean = []
for row in rows:
    if row['value'] != '' and float(row['value']) > 0:
        clean.append({'trial': row['trial'], 'value':
float(row['value'])})

# Add a log transformation
for row in clean:
    row['log_value'] = math.log(row['value'])

# Explore
print(len(rows), 'rows loaded,', len(clean), 'kept')
print(clean[:3])
```

Extract to Module (reusable):

```
# In src/data_processing.py

import csv
import math


def load_data(filepath):
    """
    Load experimental data from a CSV file.

    Parameters
    ----------
    filepath : str or Path
        Path to CSV file with columns 'trial' and 'value'.

    Returns
    -------
    list of dict
        One dictionary per row, with all values as strings.
    """
    with open(filepath, newline='') as f:
        return list(csv.DictReader(f))


def clean_data(rows):
    """
    Clean data by removing missing values and invalid entries.

    Parameters
    ----------
    rows : list of dict
        Raw rows as returned by load_data.

    Returns
    -------
    list of dict
        Cleaned rows with numeric values and a log-transformed value.
    """
    cleaned = []
    for row in rows:
        # Remove missing values
        if row['value'] == '':
            continue
        value = float(row['value'])
        # Filter invalid values
        if value <= 0:
            continue
        # Add log transformation
        cleaned.append({
            'trial': row['trial'],
            'value': value,
```

```
                'log_value': math.log(value),
            })
    return cleaned


def load_and_clean(filepath):
    """
    Load and clean data in one step.

    Parameters
    ----------
    filepath : str or Path
        Path to CSV file.

    Returns
    -------
    list of dict
        Cleaned data ready for analysis.
    """
    rows = load_data(filepath)
    return clean_data(rows)
```

Use in Notebook (clean):

```
# In notebook
from src.data_processing import load_and_clean

# Now simple one-liner
data = load_and_clean('../data/raw/experiment.csv')

# Continue with analysis
```

4. Documentation

README.md:

```
# My Research Project

Brief description of the project.

## Installation

```bash
git clone https://github.com/username/myproject.git
cd myproject
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```
```

Usage

```
from src.data_processing import load_and_clean
from src.analysis import run_analysis

# Load data
data = load_and_clean('data/raw/experiment.csv')

# Run analysis
results = run_analysis(data)
```

Data

Data files are located in `data/`:

- `raw/`: Original data files
- `processed/`: Cleaned data

Notebooks

Analysis notebooks in `notebooks/`:

1. `01_exploration.ipynb`: Initial data exploration
2. `02_analysis.ipynb`: Statistical analysis
3. `03_reports.ipynb`: Summary tables and reports

Citation

If you use this code, please cite:

> Your Name (YYYY). Project Title. GitHub: username/myproject

License

MIT License - see LICENSE file

Reproducible Research Checklist

- [ ] Code in version control (Git)
- [ ] Dependencies documented (requirements.txt / environment.yml)
- [ ] Data sources documented
- [ ] Analysis steps documented (notebooks or scripts)
- [ ] Random seeds set for reproducibility
- [ ] Software versions recorded
- [ ] README explains how to reproduce
- [ ] Figures can be regenerated from code

Publishing Your Research

Code Archival

Zenodo + GitHub Integration:

1. Go to https://zenodo.org/
2. Login with GitHub
3. Enable repository
4. Create GitHub release
5. Zenodo automatically archives and provides DOI

Benefits:

- Permanent DOI for citing code
- Immutable archive
- Discoverable

Data Sharing

Options:

- **Zenodo**: General purpose, DOI
- **Figshare**: Figures and data, DOI
- **OSF** (Open Science Framework): Projects, DOI
- **Dryad**: Biological data, DOI
- **Domain-specific**: GenBank, PDB, etc.

Notebooks as Supplements

Binder: Make notebooks executable

1. Push notebook to GitHub
2. Go to https://mybinder.org/
3. Enter repository URL
4. Share generated link

Users can run your notebook in browser without installing anything!

Continuous Integration

Services such as GitHub Actions can run your checks automatically whenever you push. Automated testing is covered later in the course.

Collaboration Best Practices

1. **Clear contribution guidelines**: CONTRIBUTING.md file
2. **Issue templates**: Help users report bugs effectively
3. **Pull request template**: Standardize PR descriptions
4. **Code review**: At least one review before merging
5. **Protected main branch**: Require PRs and reviews
6. **Semantic versioning**: v1.0.0 → v1.0.1 (patch), v1.1.0 (minor), v2.0.0 (major)

7. **Changelog**: Document changes between versions

Time-Saving Tips

1. **Aliases**: Create shortcuts for common commands

   ```
   # In ~/.bashrc or ~/.zshrc
   alias ga='git add'
   alias gc='git commit -m'
   alias gp='git push'
   alias gs='git status'
   alias jl='jupyter lab'
   ```

2. **Templates**: Create project templates

   ```
   # Create template repo on GitHub
   # Use as template for new projects
   ```

3. **Snippets**: Save common code blocks in editor
   - VS Code snippets
   - Jupyter Lab code snippets extension

4. **Makefile**: Automate common tasks

   ```
   .PHONY: run clean install

   install:
       pip install -r requirements.txt

   run:
       python scripts/run_pipeline.py

   clean:
       find . -type f -name '*.pyc' -delete
       find . -type d -name '__pycache__' -delete

   notebook:
       jupyter lab
   ```

Conclusion

Mastering these development tools and workflows will:

- **Save time** through automation and reusability
- **Reduce errors** through version control and careful review
- **Enhance collaboration** through clear processes
- **Increase impact** through reproducibility and sharing
- **Advance science** through open, transparent research

The investment in learning these tools pays dividends throughout your research career. Start small, adopt incrementally, and gradually incorporate these practices into your daily workflow.

Additional Resources

Official Documentation

- **Jupyter**: https://jupyter.org/documentation
- **Git**: https://git-scm.com/doc
- **GitHub**: https://docs.github.com/
- **Python venv**: https://docs.python.org/3/library/venv.html
- **Conda**: https://docs.conda.io/
- **Markdown**: https://www.markdownguide.org/

Tutorials

- **GitHub Skills**: https://skills.github.com/ (Interactive Git tutorials)
- **Jupyter Tutorial**: https://www.dataquest.io/blog/jupyter-notebook-tutorial/
- **Real Python Git**: https://realpython.com/python-git-github-intro/

Books

- **Pro Git** (free): https://git-scm.com/book/en/v2
- **The Turing Way**: https://the-turing-way.netlify.app/ (Reproducible research)

Community

- **Stack Overflow**: https://stackoverflow.com/
- **Jupyter Discourse**: https://discourse.jupyter.org/
- **GitHub Community**: https://github.community/

This handout is part of the "Introduction to Scientific Programming" course at CNC-UC, University of Coimbra. For questions or clarifications, please contact the course instructor. **Document Version**: 1.0
License: CC BY 4.0